



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 4

Team No.: 03

Project Title: Process Terminating Messages

Team Members

Roll Number	Student Name	Signature
2520030355	Ch. Suraj Reddy	
2520030213	Kevin Josh	
2520030387	Vanka Vijay	

1. Abstract

Process Terminating Messages is a system-level concept that focuses on the communication and notification mechanisms used when a process is terminated in an operating system. A process may terminate normally after completing its assigned task or may be terminated due to errors, resource limitations, user actions, or system-level requirements. When a process terminates, it is important to provide appropriate information about the reason and status of termination to users, system administrators, or other processes.

The proposed system demonstrates how terminating messages can be generated and handled during the process life cycle. It can display messages such as successful completion, abnormal termination, termination by the user, or termination due to an error. The system can use operating-system process management concepts such as process IDs, parent and child processes, exit status, signals, and system calls to monitor and communicate process termination events.

This project helps in understanding how operating systems manage processes and how termination information is transferred between processes and the operating system. It also improves error identification, process monitoring, and system reliability by providing meaningful termination messages. The concept can be useful in operating-system monitoring tools, server applications, background services, and debugging systems where knowing why and when a process terminated is important.

2. Problem Statement

In an operating system, processes may terminate for several reasons, such as successful completion, execution errors, user intervention, resource limitations, or system-generated signals. However, users and administrators may not always receive clear information about why a process was terminated. This can make it difficult to monitor processes, identify errors, and understand system behavior.

The **Process Terminating Messages** project aims to develop a simple system that detects process termination and displays meaningful messages indicating the termination status and reason. The system helps users understand the process life cycle and provides useful information for process monitoring and debugging.

3. Objectives

1. To understand the **process life cycle** in an operating system.
2. To identify when a process is **terminated**.
3. To display appropriate messages when a process terminates.
4. To differentiate between **normal and abnormal termination**.
5. To understand the use of **process IDs (PID)** and parent-child processes.
6. To study process termination mechanisms such as **exit status and signals**.
7. To improve process monitoring and error identification.
8. To provide users with clear information about the **reason and status of process termination**

4. Proposed Methodology

The proposed methodology for the **Process Terminating Messages** project focuses on monitoring the life cycle of a process and generating meaningful messages when the process terminates. First, a process is created and its details, such as Process ID (PID) and Parent Process ID (PPID), are recorded. The process then performs a predefined task while its execution is monitored. When the process finishes or is terminated, the system checks its exit status or termination signal to determine whether it ended normally or abnormally. Based on the termination condition, the system generates an appropriate message, such as successful completion, termination due to an error, user termination, or signal-based termination. Finally, the termination message and relevant process information are displayed to the user. Different termination scenarios are tested to ensure that the system correctly identifies the termination reason and provides accurate messages.

5. Operating Systems Concepts / Linux APIs Used

(List the relevant Operating Systems concepts, Linux system calls, POSIX APIs, or mechanisms that will be used.)

OS Concept / Linux API / System Call	Purpose in the Project
fork()	Creates a child process for demonstrating process management.
wait() / waitpid()	Allows the parent process to wait for the child process to terminate and obtain its status.

exit()	Terminates a process and returns an exit status.
kill()	Sends signals to a process for controlled termination.
Signals	Detects and handles different process termination conditions.
Process ID (PID)	Identifies and monitors individual processes during execution and termination.

6. Individual Contribution

(Clearly specify the responsibility of each team member in the project.)

Roll Number	Student Name	Individual Responsibility
2520030355	Ch. Suraj Reddy	Process creation, implementation of fork() and exec()
2520030213	Kevin Josh	Process termination, signals, and wait() / waitpid()
2520030387	Vanka Vijay	Termination message generation, testing, and documentation

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system used to develop and execute the process termination program.
C / C++	Programming language used to implement process creation, termination, signals, and system calls.
GCC Compiler	Used to compile and execute the C/C++ source code.
Linux Terminal	Used to run commands, execute programs, and monitor processes.
VS Code	Used for writing, editing, and managing the project source code.

8. Expected Outcome

The project is expected to successfully demonstrate **process termination and terminating messages** in a Linux/Ubuntu environment. The system will create and execute processes, monitor their execution, and detect when they terminate. It will display clear messages indicating whether a process completed normally, terminated due to an error, or was terminated by a signal or user action. The project will also demonstrate the use of Linux system calls such as fork(), wait(), exit(), and kill(). Overall, the project is expected to provide a practical understanding of **process management, process termination, signals, and exit status in operating systems**.

9. GitHub Repository

GitHub Repository Link: https://github.com/SurajReddy469/OSSP_Team3/tree/main/Team-3/Process%20Terminating%20Messages_PROJECT

Repository Created: _____

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: M. Raghupati

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____